



Xi'an Jiaotong-Liverpool University

西交利物浦大學

Software Testing

CPT202-

Software Engineering Group Project

Reminder

for software development,
please **use** the programming
languages, methods, tools

which have been taught or
will be **taught in this module**

Overview

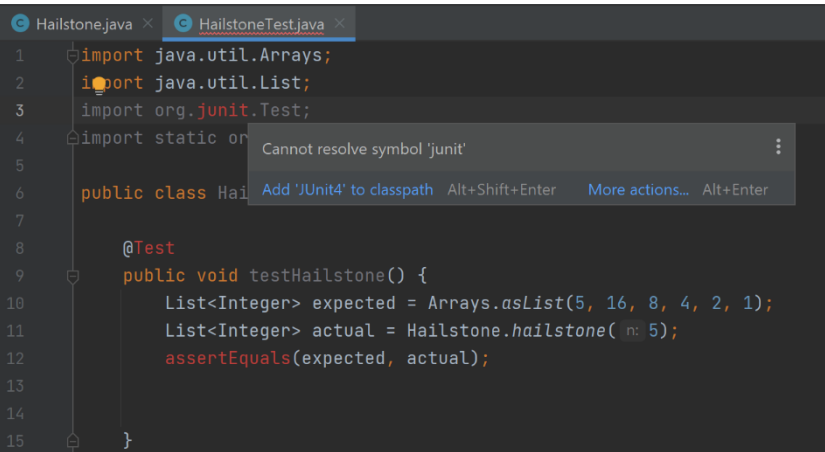
Part 1: Testing basics and a discussion

Part 2: The acceptance criteria in your PBI, related to assignment 1

Part 3: Programing tasks for software testing

- Revisit the tasks from the last semester
 - Additional tasks
-

CPT105, CPT111, CPT 204, CPT203



```
1 import java.util.Arrays;
2 import java.util.List;
3 import org.junit.Test;
4 import static org.junit.Assert.*;
5
6 public class HailstoneTest {
7
8     @Test
9     public void testHailstone() {
10         List<Integer> expected = Arrays.asList(5, 16, 8, 4, 2, 1);
11         List<Integer> actual = Hailstone.hailstone(n: 5);
12         assertEquals(expected, actual);
13     }
14 }
15
```

Software Testing

- ✓ Testing Basics
- ✓ Development testing
- ✓ Release testing
- ✓ User testing

Refer to CPT203 software testing P1

Software Testing - Junit Testing

- ✓ Unit Testing and JUnit
- ✓ Assertion Methods
- ✓ JUnit Test Cycle and Annotation

Refer to CPT203 software testing P2

- Unit testing
- Integration testing
- Acceptance testing

NetBeans



IntelliJ



JUnit



- Testing of an individual software unit
 - class, function
 - Focus on the functions of the unit
 - functionality, correctness, accuracy
 - A test is NOT a unit test if:
 - It talks to the database
 - It communicates across the network
 - It touches the file system
 - It can't run at the same time as any of other unit tests
 - The special operations to the environment (such as editing config files) to run it
-

Xi'an Jiaotong-Liverpool University
西交利物浦大學

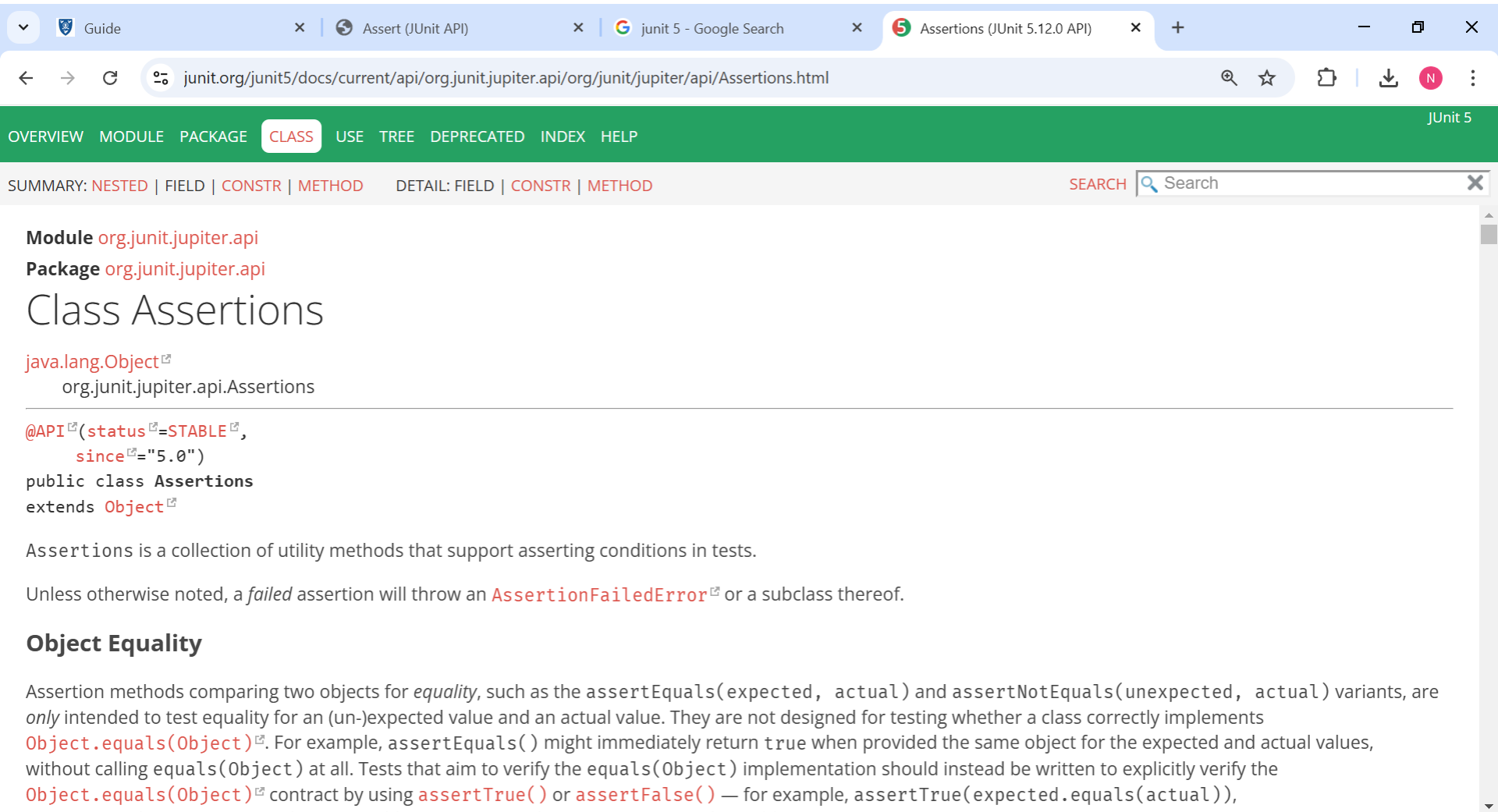
```
package Test;

import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.Disabled;
import org.junit.jupiter.api.Test;

public class Calculator {
    public static int add(int x,int y) {
        return x+y;
    }
}

Annotation
@Test
Name of test
public void CalculatorTest(){
    Assertions.assertEquals(2, Calculator.add(1, 1));
}
Method Expected Value Actual value
```

junit 5: Assertion



The screenshot shows a web browser with multiple tabs. The active tab is titled 'Assertions (JUnit 5.12.0 API)'. The address bar shows the URL: junit.org/junit5/docs/current/api/org.junit.jupiter.api/org/junit/jupiter/api/Assertions.html. The page has a green navigation bar with links: OVERVIEW, MODULE, PACKAGE, CLASS (highlighted), USE, TREE, DEPRECATED, INDEX, HELP. Below the navigation bar is a search bar and a summary section with links: NESTED, FIELD, CONSTR, METHOD, and a detail section with links: FIELD, CONSTR, METHOD. The main content area shows the class `org.junit.jupiter.api.Assertions` which extends `java.lang.Object`. It includes the `@API` annotation with `status=STABLE` and `since="5.0"`. The class is defined as `public class Assertions extends Object`. A paragraph explains that Assertions is a collection of utility methods for asserting conditions in tests. Another paragraph states that unless otherwise noted, a failed assertion will throw an `AssertionFailedError` or a subclass thereof. A section titled 'Object Equality' explains that assertion methods like `assertEquals` and `assertNotEquals` are intended to test equality for an (un-)expected value and an actual value. They are not designed for testing whether a class correctly implements `Object.equals(Object)`. Tests that aim to verify the `equals(Object)` implementation should instead be written to explicitly verify the `Object.equals(Object)` contract by using `assertTrue()` or `assertFalse()` — for example, `assertTrue(expected.equals(actual))`.

Module `org.junit.jupiter.api`
Package `org.junit.jupiter.api`

Class Assertions

`java.lang.Object`
`org.junit.jupiter.api.Assertions`

```
@API(status=STABLE,  
      since="5.0")  
public class Assertions  
    extends Object
```

Assertions is a collection of utility methods that support asserting conditions in tests.

Unless otherwise noted, a *failed* assertion will throw an `AssertionFailedError` or a subclass thereof.

Object Equality

Assertion methods comparing two objects for *equality*, such as the `assertEquals(expected, actual)` and `assertNotEquals(unexpected, actual)` variants, are *only* intended to test equality for an (un-)expected value and an actual value. They are not designed for testing whether a class correctly implements `Object.equals(Object)`. For example, `assertEquals()` might immediately return `true` when provided the same object for the expected and actual values, without calling `equals(Object)` at all. Tests that aim to verify the `equals(Object)` implementation should instead be written to explicitly verify the `Object.equals(Object)` contract by using `assertTrue()` or `assertFalse()` — for example, `assertTrue(expected.equals(actual))`,

Checked on 5 March 2025. This webpage lists all methods:

<https://junit.org/junit5/docs/current/api/org.junit.jupiter.api/org/junit/jupiter/api/Assertions.html>

More examples JUnit 5 Assertions

```
//Java file
public class MathOperations {

    public int add(int a, int b) {
        return a + b;
    }

    public int subtract(int a, int b) {
        return a - b;
    }

    public boolean isPositive(int number) {
        return number > 0;
    }

    public boolean isNegative(int number) {
        return number < 0;
    }

    public String getHello() {
        return "Hello";
    }

    public int[] getArray() {
        return new int[]{1, 2, 3};
    }

    public String getHelloString() {
```

```
//JUnit file
import static org.junit.jupiter.api.Assertions.*;

import org.junit.jupiter.api.Test;

class AssertionExamplesTest {

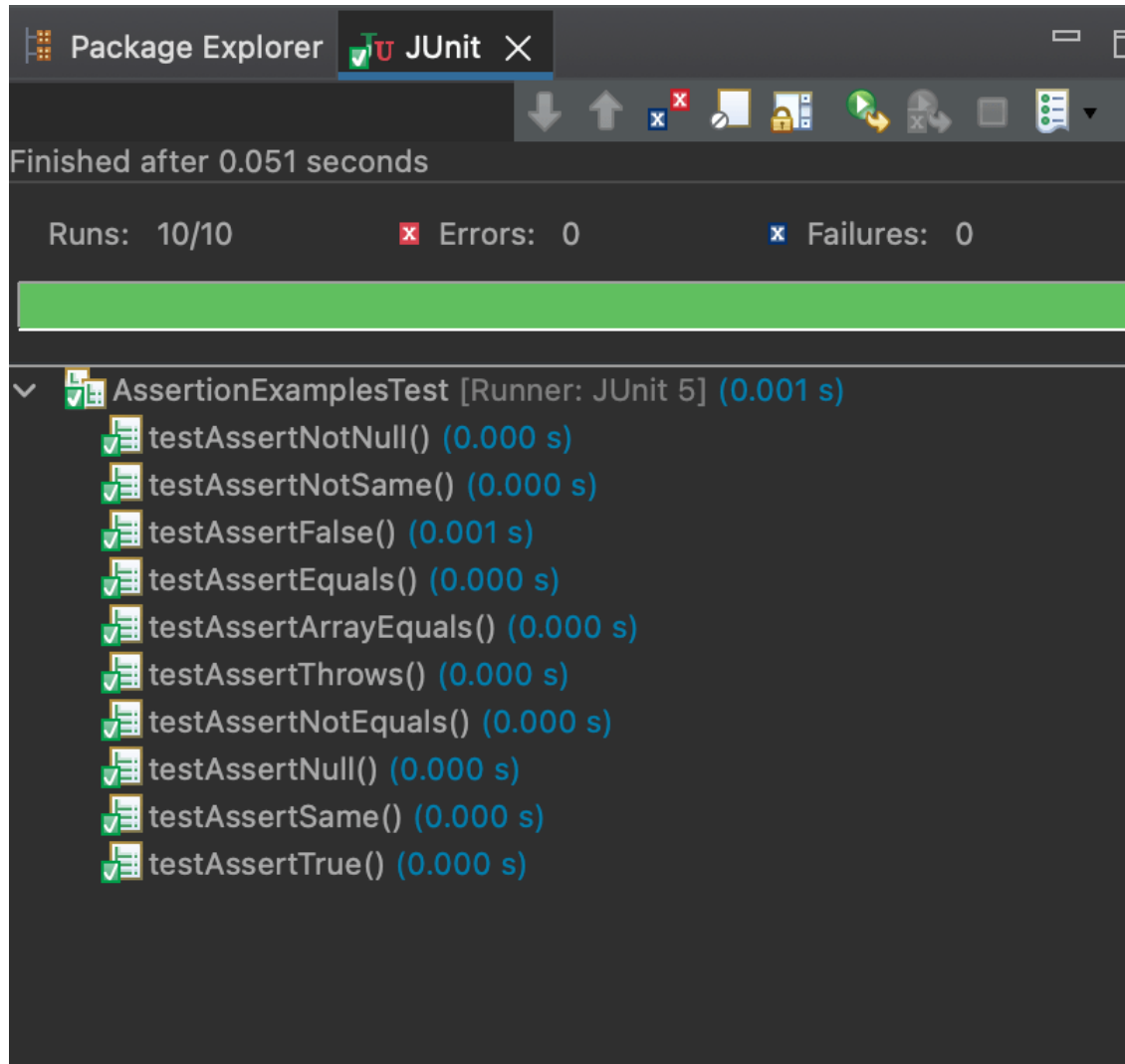
    private MathOperations mathOperations = new
    MathOperations();

    // assertEquals
    @Test
    void testAssertEquals() {
        int expected = 5;
        int actual = mathOperations.add(2, 3);
        assertEquals(expected, actual);
    }

    // assertNotEquals
    @Test
    void testAssertNotEquals() {
        int expected = 5;
        int actual = mathOperations.add(2, 2);
        assertNotEquals(expected, actual);
    }

    // assertTrue
    @Test
```

More examples JUnit 5 Assertions



NOTE: JUnit versions

2022

```
import static org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.RepeatedTest;
import org.junit.jupiter.api.Test;
```

```
class CalculatorTest {
```

```
    Calculator calculator; (1 mark)
```

//More than one way
to implement it

```
@RepeatedTest(5)          (1 mark)
@DisplayName("Simple multiplication should work") (1 mark, any output is accepted)
void testMultiply() {
    Assertions.assertEquals(20, calculator.multiply(4, 5),
        "Regular multiplication should work");
}
(4 marks, 2 for Assertions.assertEquals , 2 for the function parameters )
}
```

2023

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.RepeatedTest;
import static org.junit.Assert.assertEquals;
```

```
class CalculatorTest {
```

```
    Calculator calculator = new Calculator();
```

```
// The requirement says
// to run 5 times
// which annotation to choose?
```

```
@RepeatedTest(5)          // important
@DisplayName("Simple multiplication should work") // example text here
void testMultiply() {
```

```
    assertEquals(20, calculator.multiply(4, 5),
        "Regular multiplication should work");
```

```
// important: 1. use the right assert???() function ; 2. function parameters correct
}
}
```

```
public class assertSame {  
    @Test  
    public void AS() {  
        String s1 = new String("Hello");  
        String s2 = new String("Hello");  
        Assertions.assertSame("Hello", "Helloo");  
    }  
}
```

```
@Test  
public void AT() {  
    Assertions.assertTrue(5 < 4);  
}
```

```
public class assertArrayE {  
    @Test  
    public void AAE(){  
        int[] a1 = { 2, 3, 5, 7 };  
        int[] a2 = { 3, 3, 5, 7 };  
        Assertions.assertArrayEquals(a1, a2, "two arrays are equal");  
    }  
}
```

```
@Test  
void testConvertToDecimalInvalidDenominator() {  
    Assertions.assertThrows(Exception.class,  
        () -> MathTools.convertToDecimal(3, 0),  
        "An Illegal Argument Exception is expcted");  
}
```

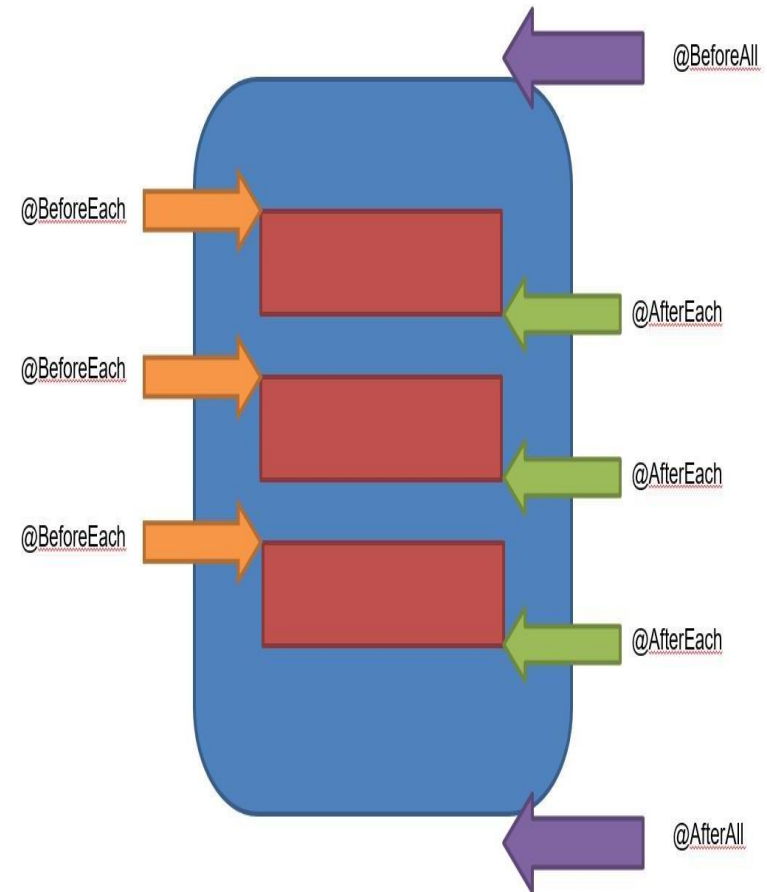
The list of methods

Assertion	Description	Example
<i>assertEquals</i> (expected, actual)	Asserts that two values are equal.	<code>assertEquals(5, 3 + 2);</code>
<i>assertNotEquals</i> (unexpected, actual)	Asserts that two values are not equal.	<code>assertNotEquals(4, 3 + 2);</code>
<i>assertTrue</i> (condition)	Asserts that a condition is true.	<code>assertTrue(5 > 3);</code>
<i>assertFalse</i> (condition)	Asserts that a condition is false.	<code>assertFalse(3 > 5);</code>
<i>assertNull</i> (object)	Asserts that an object is null.	<code>assertNull(null);</code>
<i>assertNotNull</i> (object)	Asserts that an object is not null.	<code>assertNotNull("Hello");</code>
<i>assertArrayEquals</i> (expected, actual)	Asserts that two arrays are equal.	<code>assertArrayEquals(new int[] {1, 2}, new int[] {1, 2});</code>
	Asserts that two	

In the test life cycle, we will primarily need to have some annotated methods to setup and cleanup the test environment or test data on which the tests run.

In JUnit, by default, for each test method – a new instance of test is created.

1. **@BeforeAll** and **@AfterAll** annotations – clear by their name – should be called only once in the entire tests execution cycle. So they must be declared **static**.
2. **@BeforeEach** and **@AfterEach** are invoked for each instance of test so they should not be static.



Annotation list: <https://junit.org/junit5/docs/current/user-guide/>

A verdict is the result of executing a single test case.

Pass

- The test case execution was completed
- The function being tested performed as expected

Fail

- The test case execution was completed
- The function being tested did not perform as expected

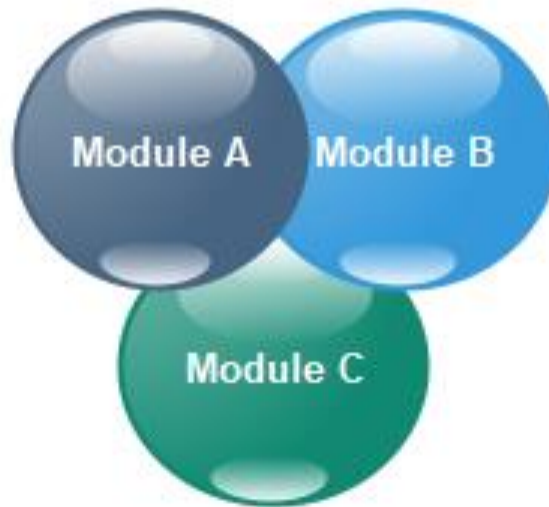
Error

- The test case execution was not completed, due to
 - an unexpected event, exceptions, or
 - improper set up of the test case, etc.
-

Unit Testing and Integration Testing



Tested in Unit Testing



Under Integration Testing

- The meaning of Integration testing is quite straightforward- *Integrate/combine the unit tested module one by one and test the behavior as a combined unit.*
 - The goal of integration testing is to check the correctness of communication/relationship among different modules/units.
 - Once all the components or modules are working independently, then we need to check the **data flow** between the dependent modules is known as integration testing.
-

excise: An online shopping website



Please think the key functions involved in ?

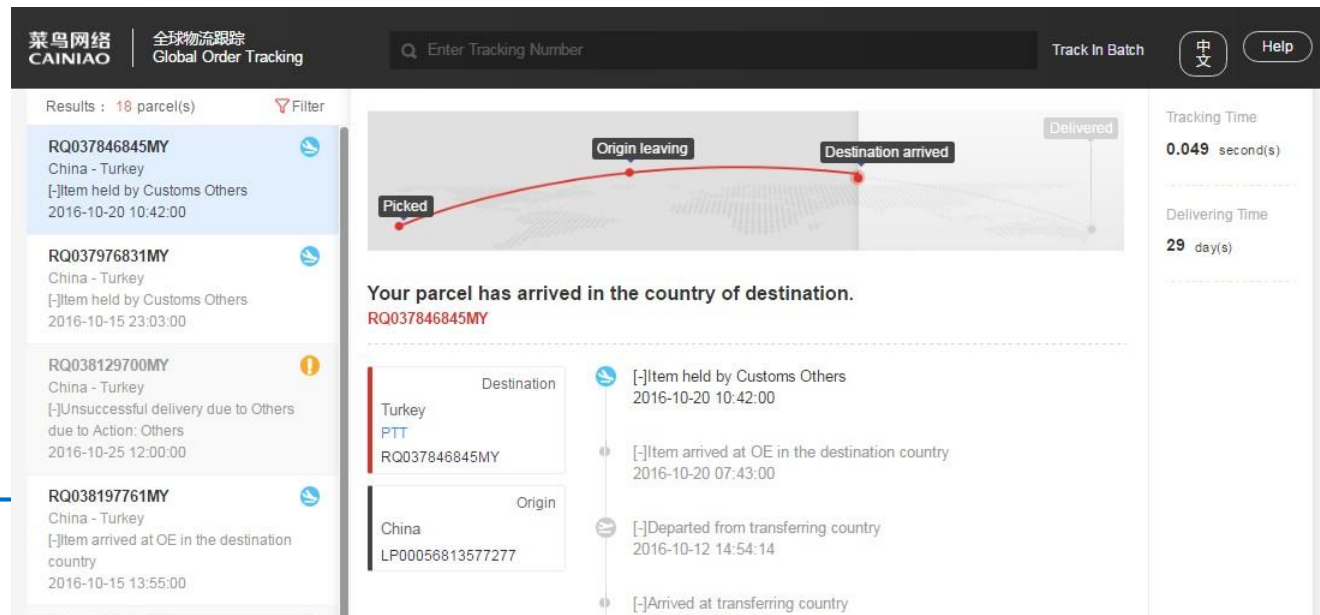
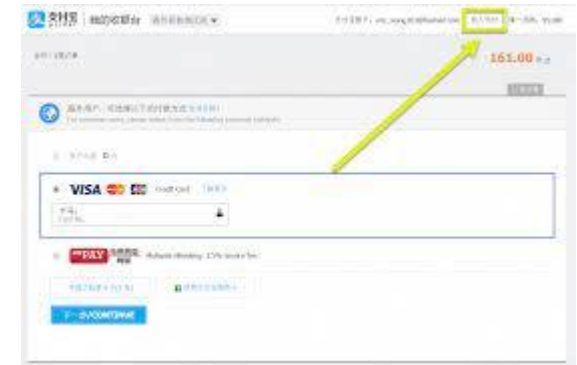
- *User registration*
- *Authentication/Login*
- *Any more?*



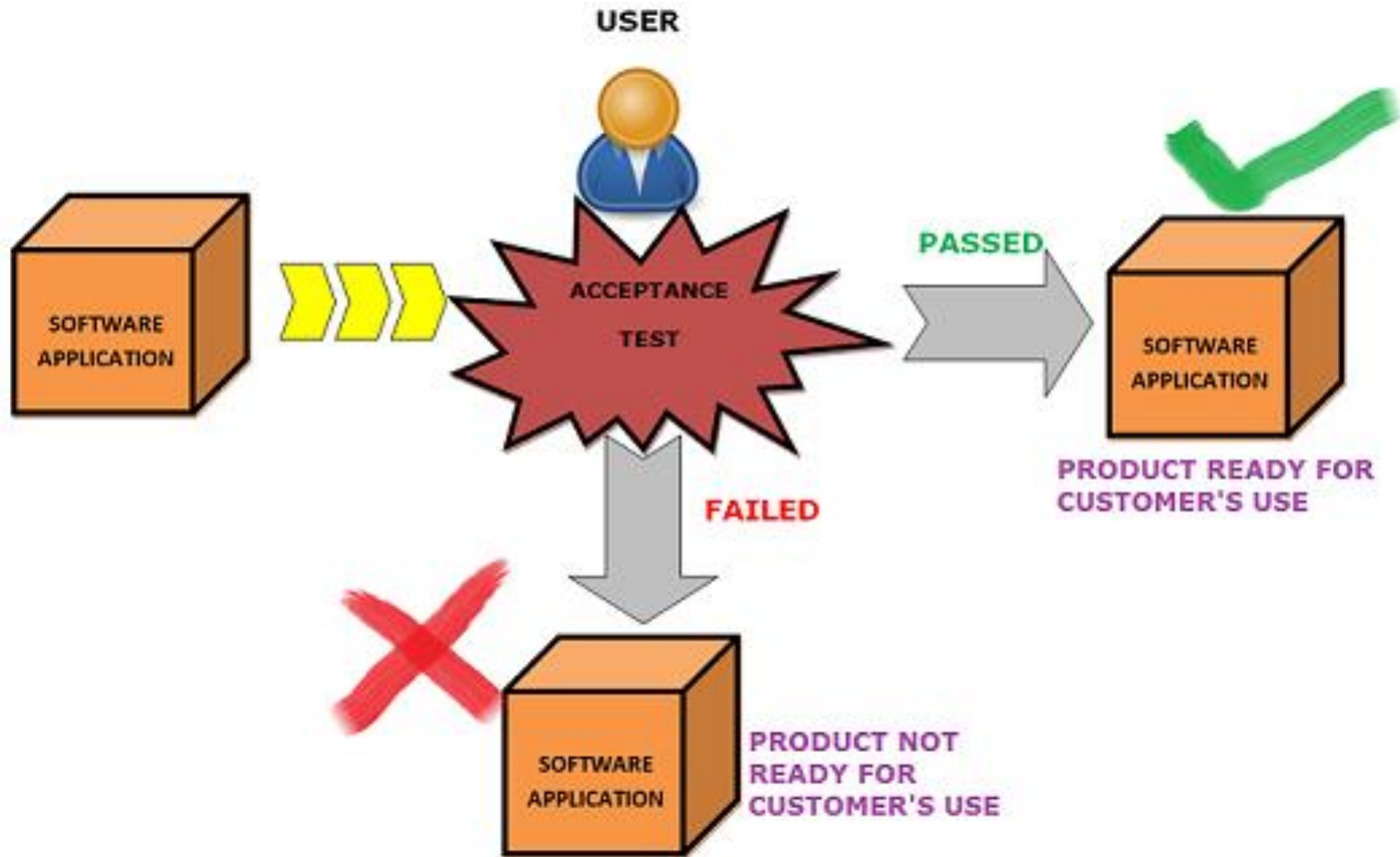
Example: An online shopping website

Function/unit Involved:

- User registration and Authentication/Login
- Product Catalogue
- Shopping Cart
- Billing
- Payment gateway integration
- Shipping and Package Tracking
-



Acceptance Testing



- Acceptance testing is a quality assurance (QA) process that determines to what degree an application meets **end users'** approval.
- Depending on the organization, acceptance testing might take the form of beta testing, application testing, field testing or end-user testing.

The acceptance criteria in your PBI



Report your test case/data

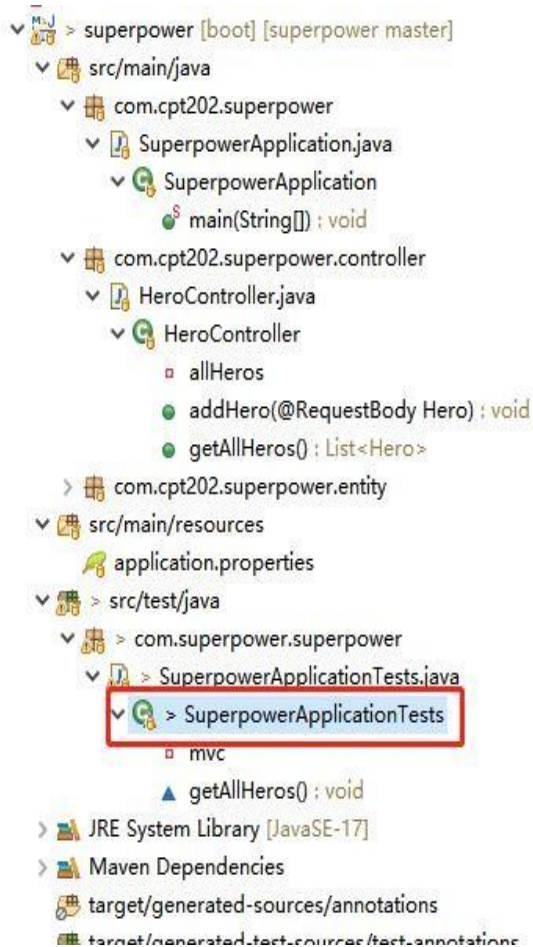
Test Name/ID	
Precondition (if any)	
Input	
Output	
Expected	
Actual Result	
Test process	{pass, fail, error}

Test case report - example

Test Case Template

Project Name	Old Fashion Oven					
Test Case ID		Designer				
Module Name	Panel operation	Tester				
Test Title	Overall run					
Description	Testing the overall run of the oven using the front panel					
Precondition	Power socket is plugged					
Dependencies						
Step	Action	Test Data	Expected Result	Actual Result	Status	Notes
1	power on		display time			
2	full power		power = 600			
3	timer		time set			
4	input number		new time set			
5	close door		Ready			
6	start		oven in operation			
7	cancel		display time			
Postcondition						
The oven go back to waiting state without completing the countdown of the time set						

- Since you are working with Spring Boot and MVC



```
15
16
17 @SpringBootTest(classes=HeroControl
18 class SuperpowerApplicationTests {
19
20
21 @Autowired
22 private MockMvc mvc;
23
24 @Test
25 void getAllHeros() throws E
26     RequestBuilder request :
27     MvcResult result = mvc.
28     Assertions.assertEquals
29 }
30
31 }
32
```


Exercises on Junit test

1. Re-visit the testing examples.
2. Note Junit versions
3. An online tutorial. Please try it



Next steps

Students: Work on the Junit exercises.
Download the required software for Junit
